Argustack Use Cases & Examples

Version 0.1.12 | March 2026

Real-world scenarios showing how teams use Argustack to answer questions that normally take hours of manual cross-referencing.

Visit the project on GitHub or install from npm.

> **Note:** This documentation is actively maintained alongside the codebase. While we verify accuracy with each release, minor discrepancies may exist as features evolve. For the latest information, refer to the GitHub repository. Found an issue? Open a ticket.

Table of Contents

1. The Cross-Referencing Problem
2. For Project Managers
3. For Team Leads
4. For Developers
5. For CTOs & Engineering Directors
6. For QA & Release Managers
7. For Onboarding
8. Example Conversations

1. The Cross-Referencing Problem

Every project has the same pain: information is scattered across multiple tools.

You want to know	You need to check
Was this ticket implemented?	Jira + Git + GitHub
Who reviewed the code for this feature?	GitHub PRs + Jira assignees
Why did this deploy break?	Git commits + Jira tickets + PR reviews
How long does this team actually take on bugs?	Jira time tracking + Git commit history
What shipped in the last release?	GitHub releases + PR links + Jira tickets

Without Argustack: Open 3-4 tools, manually search each one, cross-reference by ticket key, piece together the story. Takes 15-30 minutes per question.

With Argustack: Ask Claude in natural language. Get a complete, cross-referenced answer in seconds.

2. For Project Managers

Was a ticket implemented as described?

> "Show me the full timeline for PROJ-456 — from creation to completion"

What you get:

- When the ticket was created and by whom
- Every status change (To Do → In Progress → Code Review → Done)
- Every reassignment between developers
- All Git commits with code changes and file diffs
- PR creation, review comments, approval, and merge
- Total elapsed time from start to finish

Why it matters: One question replaces opening Jira, GitHub, and Git separately. You see the complete story of how a ticket went from idea to production.

Sprint health check

> "Show me issue stats for project PROJ — breakdown by status and assignee"

What you get:

- Total issues by status (Open, In Progress, Done, Blocked)
- Breakdown by type (Bug, Story, Task)
- Issues per assignee with their current status distribution
- Unassigned issues that need attention

Find stuck tickets

> "Search for issues in PROJ that are 'In Progress' and assigned to any developer"

> "Now cross-reference: which of these have no commits in the last 2 weeks?"

Why it matters: Identifies tickets that are nominally "in progress" but have no actual code activity — early warning for blockers.

3. For Team Leads

Sprint estimation

> "How long will it take Sarah to implement a payment webhook integration?"

What you get (with Jira API + Git data):

- Similar completed tasks with actual hours
- Sarah's personal speed coefficient based on her full history
- Bug overhead prediction (how much extra time bugs will add)
- Two estimates: pure development time and real cost including bug aftermath
- Component familiarity factor — has Sarah worked in this area before?

What you get (with CSV data only):

- Similar tasks and their resolution timeline in business days
- Min-max range showing fastest and slowest similar completions
- Clear labeling that this is lead time (backlog + dev + review), not active work

Who knows this code?

> "Show commit stats filtered by file path 'src/payments/' — who are the top authors?"

What you get:

- Top committers to the payments module
- Number of commits and lines changed per person
- Most recently active developer in that area

Why it matters: When assigning a new task, you can find the developer most familiar with that part of the codebase.

Code review coverage

> "Find all PRs merged in the last month that had no reviews"

```
query_prs with sql: SELECT * FROM pull_requests
WHERE merged_at > NOW() - INTERVAL '30 days'
AND number NOT IN (SELECT DISTINCT pr_number FROM pr_reviews)
```

Why it matters: Identifies code that went to production without peer review.

4. For Developers

Understand the context behind a ticket

> "Show me PROJ-789 with full description, comments, and changelog"

What you get:

- Complete ticket description with all formatting
- Every comment in chronological order
- All field changes (priority bumps, re-assignments, scope changes)
- All custom fields (story points, components, sprints)
- Linked issues (blocks, is blocked by, duplicates)

Find related code changes

> "What commits mention PROJ-789? Show me the file changes"

What you get:

- Every commit that references the ticket key in its message
- For each commit: files changed, lines added/deleted
- Author and date for each commit

Find similar past issues

> "Search for issues similar to 'users getting timeout on password reset'"

Argustack's hybrid search combines full-text keyword matching with AI vector similarity. A ticket titled "Authentication flow hangs during credential recovery" would match even though it uses completely different words. Works without embeddings (text-only mode), and gets better with `argustack embed`.

Check PR review status

> "Was PROJ-789 reviewed? Who approved it?"

What you get:

- All PRs linked to this ticket
- Review status: approved, changes requested, commented
- Reviewer names and their decisions
- Whether the PR was merged without approval

5. For CTOs & Engineering Directors

Team velocity analysis

> "Show me commit stats for the last quarter — top authors, most changed files, activity by day"

What you get:

- Total commits and linked issues
- Most active contributors
- Files that change most often (hotspots — potential tech debt)
- Activity patterns by day of week

Release audit

> "What tickets were part of the v2.5.0 release?"

Flow:

1. `query_releases` finds the release with tag v2.5.0
2. Release body contains PR references
3. `query_prs` shows each PR's details
4. PR titles contain Jira issue keys
5. `get_issue` shows the original business requirement

Result: Complete traceability from business requirement → code → review → release.

Bug cost analysis

> "Estimate: how much extra time do bugs typically cost us after a Story is implemented?"

Using the estimate tool with historical data, you can see:

- Average bug overhead as percentage of development time
- Which components generate the most bugs
- Which developers have the highest/lowest bug rates
- Cost comparison: estimate vs actual vs actual+bugs

6. For QA & Release Managers

Pre-release checklist

> "Show me all PRs merged into main since the last release. Which ones have linked Jira tickets?"

Identifies:

- PRs with proper ticket references (traceable)
- PRs without ticket references (undocumented changes)
- Ticket types that made it into this release (features vs bugs vs chores)

Regression investigation

> "Show me the timeline for PROJ-321 — it was marked as Done but the bug came back"

The timeline reveals:

- Original fix: commit, PR, review, merge
- When it was reopened
- New commits attempting to fix again
- Whether the second fix was reviewed

Release notes generation

> "List all PRs merged since tag v2.4.0, grouped by type (feature, bugfix, chore)"

Claude can cross-reference PR titles with Jira issue types to auto-categorize changes and draft release notes.

7. For Onboarding

New developer orientation

> "What are the most frequently changed files in the last 3 months?"

Shows the codebase hotspots — where the team spends most of its time. A new developer should start by understanding these files.

> "Who are the top 5 contributors? What areas does each person work on?"

Maps team members to their expertise areas, so the new developer knows who to ask about what.

Understanding a feature area

> "Search for all issues related to 'export' — show me the full history"

> "Now show me commits and PRs for the top 3 issues"

Gives a new team member the complete story of how a feature evolved — from original requirements through implementation to bug fixes.

8. Example Conversations

Scenario: Sprint Retrospective

PM: "How did sprint 47 go? Any surprises?"

With Argustack, ask Claude:

> "Search for issues resolved in the last 2 weeks in project PROJ. For each, show me if the actual time matched the estimate."

> "Which issues took more than 2x their original estimate? Show me their timelines."

> "What bugs were created during this sprint? Are they linked to any stories?"

Result: In 3 questions, you have a complete sprint retrospective with data, not just opinions.

Scenario: Incident Investigation

On-call engineer: "Production is broken after yesterday's deploy."

With Argustack, ask Claude:

> "Show me all commits merged yesterday"

> "For each commit, show me the linked Jira ticket and PR reviews"

> "Which of these PRs had the most file changes? Show me the diff summary"

Result: Narrow down the likely culprit in minutes instead of hours of git log and GitHub browsing.

Scenario: Stakeholder Report

CTO: "I need a status update on the payments initiative."

With Argustack, ask Claude:

> "Search for all issues with 'payment' in project PROJ. Group by status."

> "For Done issues, show me the average resolution time"

> "For In Progress issues, estimate how long they'll take based on similar past tasks"

Result: Data-driven status report generated in under a minute.

Why Argustack?

Without Argustack	With Argustack
Open Jira, search manually	Ask Claude in natural language
Switch to GitHub, find PRs	Automatic cross-referencing
Open terminal, search git log	All sources in one query
Piece together the story	Complete timeline in seconds
Estimates based on gut feeling	Data-driven predictions
15-30 minutes per question	5-10 seconds per question

All data stays on your machine. Nothing leaves localhost.

Visit the project on **GitHub** or install from **npm**.